

# LifeFlow

## Blood Donation and Emergency Matching Simulator

| OBJECT ORIENTED PROGRAMMING | SDG 3                      |
|-----------------------------|----------------------------|
| Java Swing Application      | Good Health and Well-being |

### Technical Project Report Draft

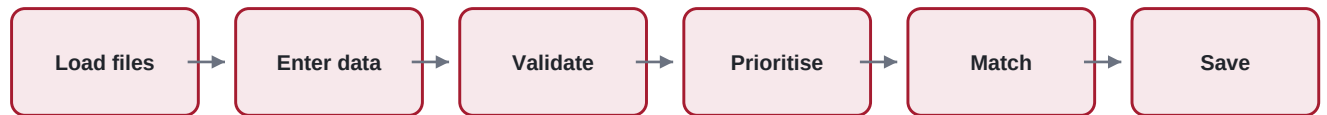
GitHub: <https://github.com/moha-r/LifeFlow>

*The private LMS submission copy must add the required team names, student IDs, class code, programme, and NRIC/passport numbers. Government identifiers must not be published in a public repository.*

*Educational simulation only - not for real medical or transfusion decisions.*

# Executive Summary

LifeFlow is a Java Swing desktop application that records donors, manages blood units, creates regular and emergency blood requests, and processes the highest-priority request using exact ABO and Rh matching. The system combines a four-tab GUI with object-oriented design, collections, decision-making, and pipe-delimited File I/O.



## Requirements coverage

| Requirement                 | LifeFlow implementation                                     |
|-----------------------------|-------------------------------------------------------------|
| GUI-based Java application  | One Swing JFrame with four tabs                             |
| Real-world simulation       | Emergency-first queue and full-quantity matching            |
| Encapsulation               | Private fields and controlled accessors                     |
| Inheritance and abstraction | Abstract BloodRequest with two subclasses                   |
| Runtime polymorphism        | getPriority() called through BloodRequest references        |
| Collections                 | ArrayList data stores and HashMap stock summary             |
| File handling               | Three persistent pipe-delimited text files                  |
| Decision-making             | Eligibility, expiry, priority, quantity, and matching rules |

# 1. Introduction and SDG Background

Reliable access to suitable blood is an important part of healthcare delivery. Blood banks record donations, monitor stock, handle requests, and give urgent cases appropriate priority. LifeFlow models these activities through a focused desktop simulation using Java and object-oriented programming.

The application aligns with Sustainable Development Goal 3 because organised donor records, visible stock, and consistent request prioritisation illustrate how software can support health services. It does not make clinical decisions and is not a substitute for blood-bank testing or professional judgement.

## 2. Problem Statement

A basic CRUD system can store donors and units, but storage alone does not show how limited stock should be handled when requests compete. A meaningful simulation must distinguish emergency requests, reject expired or used units, require the full requested quantity, and preserve its data between sessions.

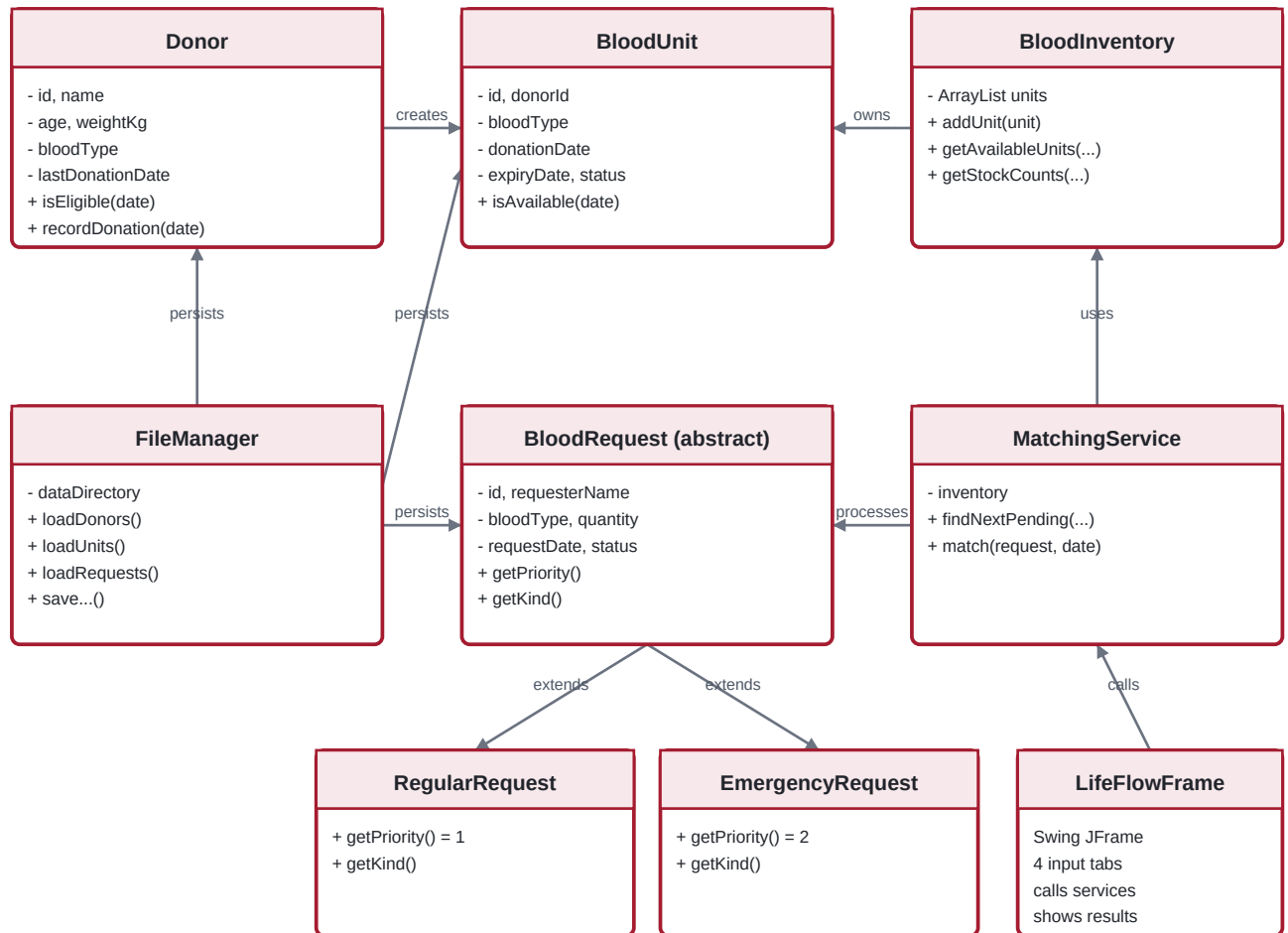
The project problem is therefore: how can a small desktop application clearly simulate blood-bank recording, prioritisation, and exact-group matching without the complexity of a hospital information system?

## 3. System Objectives

1. Register donors with identity, age, weight, blood group, and donation history.
2. Check simplified eligibility before creating a blood unit.
3. Maintain visible stock for all eight ABO and Rh groups.
4. Create regular and emergency requests with different runtime priorities.
5. Match available, non-expired units of the exact requested group.
6. Prevent partial fulfilment when the complete quantity is unavailable.
7. Save and restore all data and request subclasses using text files.

## 4. OOP Design and UML

The design separates model objects, services, persistence, and the Swing interface. BloodRequest is an abstract superclass. RegularRequest and EmergencyRequest override getPriority(), and MatchingService invokes that method through BloodRequest references. This is runtime polymorphism rather than a decorative inheritance hierarchy.



Encapsulation uses private fields and accessors. BloodInventory owns an ArrayList of BloodUnit objects and produces a HashMap stock summary. FileManager handles only text mapping, while LifeFlowFrame manages input, tables, and messages.

## 5. Implementation Details

### 5.1 Donor eligibility

For this educational simulation, a donation is accepted when the entered date is not in the future, age is 18-60, weight is at least 45 kg, and either no previous donation exists or three months have passed. These are simplified project rules and must not be used to decide real donor eligibility.

### 5.2 Request processing

`findNextPending()` loops through pending requests. A request replaces the current choice when it has a higher polymorphic priority, or the same priority and an earlier date. `match()` filters inventory by exact group, AVAILABLE status, and expiry date. Insufficient stock changes nothing; sufficient stock marks the selected units USED and the request FULFILLED.

### 5.3 Swing interface

One JFrame contains Donors, Blood Inventory, Blood Requests, and Matching tabs. Read-only sortable tables show current state. Forms validate required fields, duplicate IDs, numbers, dates, quantities, and the file delimiter. JOptionPane displays clear errors.

### 5.4 Persistence

FileManager stores three pipe-delimited files with yyyy-MM-dd dates. Missing files produce empty collections. Malformed lines report their file and line number and stop start-up, avoiding silent overwrite of unreadable data.

### Data schemas

```
donors.txt: id|name|age|weightKg|bloodType|lastDonationDate
blood_units.txt: id|donorId|bloodType|donationDate|expiryDate|status
requests.txt: id|kind|requesterName|bloodType|quantity|requestDate|status
```

## 6. Testing and Sample Outputs

Assertion-based tests run without external libraries. The verified build uses Java 17 compatibility and enables all assertions during the test run.

| Area         | Scenario                      | Expected result                     |
|--------------|-------------------------------|-------------------------------------|
| Eligibility  | Age 17 / 18 / 60 / 61         | Reject / accept / accept / reject   |
| Eligibility  | Weight 44.9 / 45 kg           | Reject / accept                     |
| Donation gap | Before / on three-month date  | Reject / accept                     |
| Inventory    | Used, expired, wrong group    | Excluded from available units       |
| Polymorphism | Emergency and regular request | Priority 2 and 1 at runtime         |
| Queue        | Emergency newer than regular  | Emergency selected first            |
| Matching     | Full quantity unavailable     | No units consumed; request pending  |
| Persistence  | Save and load both subclasses | Correct subclass and state restored |
| Bad data     | Malformed donors line         | IOException names file and line     |

### Verified automated-test output

```
javac --release 17 -Xlint:all -d out <all source and test files>
java -ea -cp out lifeflow.AllTests
All tests passed.
```

## 6.1 Swing Demo Evidence

The following screenshots use fictional demonstration data. The request table shows both subclasses and their runtime priorities; the matching result confirms that the emergency request was processed first and consumed exactly two units.

The screenshot shows the 'LifeFlow - Blood Donation and Emergency Matching Simulator' window. The 'Blood Requests' tab is active. It contains input fields for 'Request ID', 'Requester', 'Blood Type' (set to 'A\_POS'), 'Quantity', and 'Request Kind' (set to 'REGULAR'). An 'Add Request' button is at the bottom right. Below the inputs is a table with the following data:

| Request ID | Kind      | Requester        | Blood Type | Quantity | Date       | Priority | Status  |
|------------|-----------|------------------|------------|----------|------------|----------|---------|
| R001       | REGULAR   | Community Clinic | A_POS      | 1        | 2026-08-03 | 1        | PENDING |
| R002       | EMERGENCY | City Hospital ER | O_NEG      | 2        | 2026-08-03 | 2        | PENDING |

Figure 1. Regular and emergency requests before matching.

The screenshot shows the 'LifeFlow - Blood Donation and Emergency Matching Simulator' window. The 'Matching' tab is active. It contains a 'Process Next Request' button. Below the button, a message box displays the following text:

Request R002 was fulfilled successfully.  
Matched unit(s): U001, U002

Figure 2. Emergency request R002 fulfilled with units U001 and U002.

## 7. Discussion and Limitations

LifeFlow demonstrates required OOP concepts through behaviour rather than unused classes. Its focused structure is easy to test and explain, and local persistence makes the demonstration repeatable without database setup.

The system intentionally uses exact ABO/Rh matching only. It does not model crossmatching, antibodies, components, laboratory screening, clinical exceptions, user accounts, networking, or concurrent access. Eligibility rules are simplified. Text files are appropriate for a class project but not for a production blood bank.

## 8. Conclusion

LifeFlow combines a working Swing interface with a meaningful priority and matching simulation. Encapsulation protects state; inheritance, abstraction, overriding, and runtime polymorphism define request behaviour; collections manage data; and File I/O provides persistence. The result is a focused SDG 3 application suitable for an introductory OOP project.

## References

1. City University Malaysia. BIT1123/BISE2093/DIT1113 Object Oriented Programming Final Project brief.
2. Hospital Sultan Ismail, Ministry of Health Malaysia. Syarat-Syarat Penderma Darah Yang Layak. <https://jknjohor.moh.gov.my/hsi/syarat-syarat-penderma-darah-yang-layak-2/>
3. Hospital Wanita dan Kanak-Kanak Sabah, Ministry of Health Malaysia. Kriteria Penderma Darah. [https://jknsabah.moh.gov.my/hwks/en/?id=178&option=com\\_sppagebuilder&view=page](https://jknsabah.moh.gov.my/hwks/en/?id=178&option=com_sppagebuilder&view=page)
4. Ministry of Health Malaysia. Handbook on Clinical Use of Blood. <https://portal.appshtaa.moh.gov.my/images/ProfilJabatan/Transfusi/handbook.pdf>

### Private LMS submission checklist

| Required item              | Status before submission                                                            |
|----------------------------|-------------------------------------------------------------------------------------|
| Full names and student IDs | Team must add                                                                       |
| Class code and programme   | Team must add                                                                       |
| NRIC/passport numbers      | Private LMS copy only                                                               |
| GitHub URL                 | <a href="https://github.com/moha-r/LifeFlow">https://github.com/moha-r/LifeFlow</a> |
| Final screenshots          | Captured with fictional demo data                                                   |